Institut Esteve Terradas – Departament d'Informàtica

MDUAL – Node.js i eines d'IA / Bloc 1 – Fonaments de Node.js

Comprovació ràpida del Bloc 1 (20 minuts)

Data: 15/10/26 Professor: Jordi Palà

Nom i cognoms: ___ Carlos Medina Gálvez _____

Instruccions

– Sense IA: no es pot fer servir cap assistent d'IA (xatbots, Copilot o similars). Respon en aquest full.
– Quan es demana **per què**, una resposta sense explicació no puntua.
– Temps: 20 minuts. L'exercici val 10 punts.

Apartat	Què es comprova	Punts
a	Llegir codi: event loop i mètodes d'array (activitats 2 i 3)	3
b	Errors en codi asíncron (activitats 2 i 3)	2
c	Escriure una funció pura amb mètodes d'array (activitat 2)	3
d	Corregir codi amb `try/catch` (activitat 2)	2

Exercici 1 · L'inventari de l'aula

Un projecte té aquests tres fitxers. S'executa amb `node app.js` des de la seva carpeta.

material.js

```js
function disponibles(llista) {
  return llista.filter((m) => m.estat === "disponible");
}

function valorTotal(llista) {
  return llista.reduce((suma, m) => suma + m.valor, 0);
}

module.exports = { disponibles, valorTotal };
```

app.js

```js
const fs = require("node:fs");
const { disponibles, valorTotal } = require("./material");

console.log("A · Inici");
fs.readFile("data/material.json", "utf8", (err, text) => {
  if (err) return console.error("No trobo material.json");
  const llista = JSON.parse(text);
  console.log(`B · Disponibles: ${disponibles(llista).length}`);
  console.log(`C · Valor: ${valorTotal(llista)} €`);
});
process.nextTick(() => console.log("D · nextTick"));
console.log("E · Fi");
```

data/material.json

```json
[
  { "nom": "HP 14", "valor": 450, "estat": "disponible" },
  { "nom": "iPad", "valor": 0, "estat": "disponible" },
  { "nom": "BenQ MS560", "valor": 310, "estat": "prestat", "prestatA": "Pau" }
]
```

Institut Esteve Terradas – Departament d'Informàtica

MDUAL – Node.js i eines d'IA / Bloc 1 – Fonaments de Node.js

Comprovació ràpida del Bloc 1 (20 minuts)

Data: 15/10/26 Professor: Jordi Palà

a) Escriu què mostra `node app.js`, en ordre, i explica per què cada línia surt en aquell lloc. La primera fila és un exemple resolt.

Ordre	Què surt	Per què surt en aquest lloc
1r	*A · Inici*	*Codi síncron: s'executa abans que cap callback*
2n	E · Fi	També codi síncron: s'executa abans de callbacks
3r	D · nextTick	L'ordre és: Síncron => nextTick/Promises => Callbacks/Macrotasques Just després del codi síncron, es mira primer nextTick (i promeses)
4t	B · Disponibles	Primera línia del callback de fs
5è	C · Valor	Segona línia del callback de fs

b) Què mostra si **data/material.json** no existeix? I si existeix però el JSON està mal escrit? Explica la diferència.

Si no existeix, mostra l'error "No trobo material.json"

Si existeix mal escrit, mostra error en intentar fer JSON.parse

La diferència és que en el primer cas, l'error està capturat i gestionat dins del codi,

mentre que en el segon cas, l'error es llença i surt per consola cru, sense cap gestió,

aturant l'execució de tot el programa inesperadament. (peta el programa)

c) Afegeix a **material.js** una funció pura `perEstat(llista)` que compti quants elements hi ha de cada estat, i exporta-la.

Exemple d'ús

```
perEstat(llista); // { disponible: 2, prestat: 1 }  (amb les dades de material.json)
perEstat([]);     // {}
```

(vegi ../material.js)

d) Reescriu el callback d'**app.js** (línies 5 a 10) perquè, si el JSON està mal escrit, mostri «material.json no és JSON vàlid» amb `console.error` en lloc de fer petar el programa.

(vegi ../app.js)

Com de segur/a estàs de les teves respostes? ☐ 1 gens ☐ 2 poc ☐ 3 bastant ☒ **4 molt**